Aula 09 - Testes Exploratórios e Identificação de Problemas

Descrição: Aplicação de metodologias de testes exploratórios (heurísticas) para identificar falhas não mapeadas. Introdução à automação e design robusto de software com TDD (Red-Green-Refactor), Padrão AAA e o framework Pytest, visando isolamento e previsibilidade no código.

Cenário Real: O mercado ágil de desenvolvimento não perdoa a lentidão nem a falta de qualidade. Imagine que nossa equipe acabou de lançar um aplicativo de e-commerce durante a Black Friday. Os roteiros tradicionais de testes passaram com sucesso, mas, logo na primeira hora, usuários reais descobrem que ao "clicar rapidamente e duas vezes" no botão de compra, o frete fica zerado. Os roteiros fixos e mecânicos não pegaram essa falha.

É nesse momento de caos que entra a intuição afiada de um testador utilizando ***Testes Exploratórios*** guiados por heurísticas e, posteriormente, a segurança inabalável de testes automatizados com TDD para garantir que esse erro nunca mais retorne. Hoje, nós vamos unir a criatividade investigativa de um "hacker" com o rigor de um engenheiro de software.

Pergunta: Vocês já conseguiram quebrar um jogo ou encontrar um bug em um aplicativo simplesmente porque começaram a apertar os botões fora da ordem lógica esperada pela tela? Como foi essa experiência?

Testes Exploratórios e Descoberta de Erros (Error Guessing)

A. Conceito: Vamos decompor esses conceitos para que fiquem ainda mais claros. Imagine que testar um software é como explorar uma caverna desconhecida: você pode seguir um mapa rígido ou pode usar sua intuição de explorador para encontrar passagens secretas.

Aqui está uma explicação didática dividida em três pilares:

Teste Exploratório: O "Voo Livre" Estruturado
Diferente dos testes tradicionais, onde você segue uma "receita de bolo" (script), no **Teste Exploratório** você é o autor e o executor ao mesmo tempo.

- **O Ciclo Simultâneo:** Enquanto você clica em um botão, você está **aprendendo** como ele reage, **planejando** o próximo passo e **executando** o teste. Tudo acontece em frações de segundo na sua mente.
- **Por que funciona?** Em projetos ágeis, o software muda muito rápido. Se fôssemos escrever manuais para cada pequena mudança, o software já estaria obsoleto antes do teste acabar. A exploração traz **agilidade**.

Error Guessing: O "Faro" do Testador
O **Error Guessing** (Descoberta de Erro) é a nossa "bola de cristal" baseada em fatos. Não é um chute aleatório; é um palpite educado.
Sabe quando você olha para um formulário e pensa: *"Certo, se eu colar um texto de 50 parágrafos aqui, esse sistema vai travar"*? Isso é Error Guessing. Você usa:

- **Experiência:** "No último projeto, campos de data sempre davam erro no ano bissexto."
- **Heurísticas:** Atalhos mentais para focar onde o perigo mora.
- **Casos de Borda:** Em vez de testar o óbvio (o "caminho feliz"), você testa os extremos.

Caminho Feliz	Error Guessing (Casos de Borda)
Digitar um nome comum.	Deixar o campo vazio ou usar emojis.
Comprar 1 item no carrinho.	Tentar comprar -1 item ou 999.999 itens.
Clicar em "Salvar" uma vez.	Clicar em "Salvar" dez vezes seguidas bem rápido.

O Fator Humano: Curiosidade e Crítica

O que diferencia um robô de um testador exploratório são as **Soft Skills**:

- **Pensamento Analítico:** Não apenas ver que deu erro, mas tentar entender *por que* deu erro.
- **Curiosidade:** É o famoso "e se?". *"E se eu desligar o Wi-Fi bem na hora que o pagamento estiver processando?"*
- **Adaptabilidade:** Se você descobre algo estranho em uma tela, você tem a liberdade de mudar seu plano de teste na hora para investigar aquele "fio solto".

Resumindo: O Teste Exploratório é a **estratégia** de navegar pelo sistema com liberdade, enquanto o Error Guessing é a **ferramenta mental** que nos diz onde cavar para encontrar os bugs mais escondidos.

B. Visualizando:

C. Exemplo de Código:

Embora exploratório seja muito "manual", nós usamos *scripts* simples para automatizar o *Error Guessing*.

Python

```
# Simulador de Error Guessing em uma função de divisão de preços
def calcular_parcela(preco_total, numero_parcelas):
    if numero_parcelas <= 0:
        raise ValueError("Número de parcelas deve ser maior que zero.")
    return round(preco_total / numero_parcelas, 2)

# Script rápido para injetar valores bizarros (Testando heurísticas)
```

```
casos_de_borda = [
    (1000.0, 0),        # Divisão por zero
    (-50.0, 2),         # Preço negativo
    (999999999999, 12), # Valor absurdamente alto
    (100.0, -5)         # Parcelas negativas
]

for preco, parcela in casos_de_borda:
    try:
        resultado = calcular_parcela(preco, parcela)
        print(f"Sucesso: {preco} em {parcela}x = {resultado}")
    except Exception as e:
        print(f"Falha capturada (Esperada ou Bug!): {e}")
```

D. Responda:
1. Cite duas situações que você adicionaria à lista `casos_de_borda`.
2. Como o teste exploratório difere de seguir um plano de testes?
3. O que acontece com o código de exemplo se enviarmos uma *string* no lugar do preço?
4. Qual é o papel da "experiência" na técnica de *error guessing*?
5. Como você usaria IA Generativa para te dar ideias de *casos de borda*? E quais os riscos éticos de colocar código proprietário da empresa no prompt da IA?

TDD e o Ciclo Red-Green-Refactor

A. Conceito: O **TDD (Test-Driven Development)**, ou Desenvolvimento Orientado a Testes, é como construir uma casa começando pela planta e pela inspeção, antes de colocar o primeiro tijolo. Em vez de escrever o código e depois rezar para que ele funcione, você define o que é o "sucesso" antes de começar.

Aqui está o detalhamento desse ciclo que transforma a forma como escrevemos software:

O Mantra: Red-Green-Refactor
Imagine que você é um artesão. O TDD garante que cada peça que você produz seja testada individualmente antes de ser montada no conjunto. Esse processo ocorre em três etapas cíclicas:

1º. Red (Vermelho)
Nesta etapa, você escreve um teste para uma funcionalidade que **ainda não existe**.
- **O objetivo:** O teste **deve falhar**.
- **Por que fazer isso?** Para garantir que o teste realmente funciona e que ele não está passando por "acidente". Isso prova que existe um requisito que o sistema ainda não atende.

2º. Green (Verde)
Agora, você escreve o código da funcionalidade. Mas atenção: a regra aqui é o **minimalismo**.
- **O objetivo:** Fazer o teste passar o mais rápido possível.
- **O "Código Feio":** Não se preocupe com elegância, performance ou arquitetura perfeita agora. Se o teste pede que uma função some **2 + 2**, e você escrever `return 4`, o teste passará. O foco é sair da zona de falha.

3º. Refactor (Refatorar)

Com o teste brilhando em verde (sucesso), você tem uma rede de segurança para melhorar o que fez.

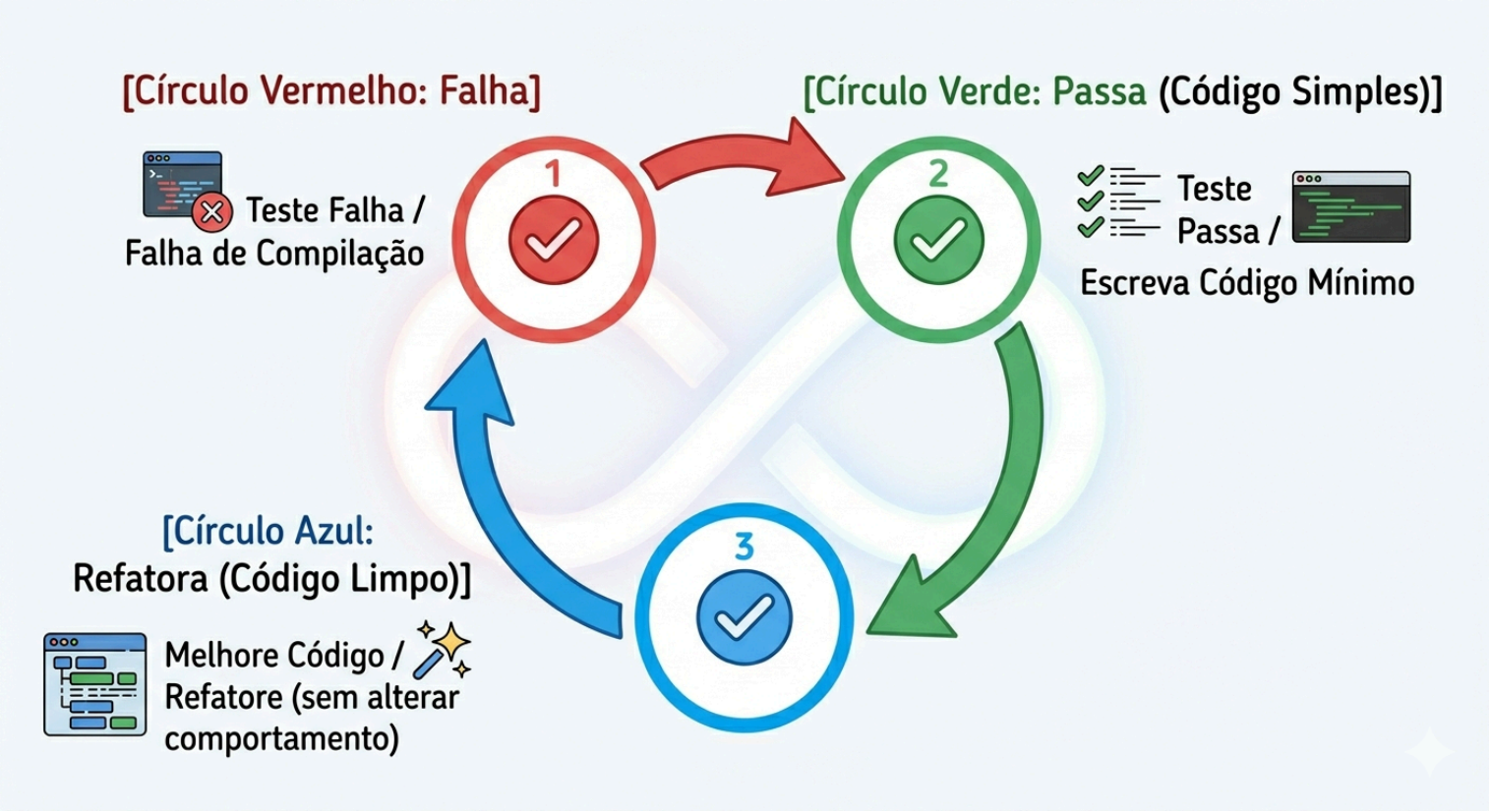
- **O objetivo:** Limpar o código, remover duplicidades, melhorar nomes de variáveis e aplicar padrões de projeto.
- **A Segurança:** Como o teste já existe, se você "fizer bobagem" enquanto limpa o código, o teste ficará vermelho imediatamente, avisando que você quebrou algo.

Por que mudar a mentalidade?
Muitos desenvolvedores sentem a tentação de pular direto para o código. No entanto, o TDD oferece vantagens estratégicas:

- **Design Emergente:** Você é forçado a pensar em como a função será usada antes de como ela será escrita. Isso gera interfaces mais simples e intuitivas.
- **Documentação Viva:** Os testes são o melhor manual de instruções que existe. Eles mostram exatamente o que o código deveria fazer em cada cenário.
- **Fim do Medo:** Sabe aquele medo de alterar um código antigo e "quebrar o sistema inteiro"? Com TDD, se você quebrar algo, um alerta vermelho acende na hora.

Analogia Prática: É como um GPS. Primeiro você diz para onde quer ir (**Red**), o GPS traça a rota mais rápida, mesmo que seja por ruas apertadas (**Green**), e depois você decide se quer ajustar o caminho para passar por uma estrada mais bonita ou evitar pedágios (**Refactor**).

B. Visualizando:



C. Exemplo de Código:
Usando `pytest` para seguir o TDD na criação de um validador de cupons.

```Python
import pytest

# 2. GREEN: O código simples que faz o teste passar
def validar_cupom(cupom):
    # 3. REFACTOR: Melhorando o código sem quebrar o teste verde
    cupons_validos = ["SENAI20", "PROMO10"]
```

```
    return cupom.upper().strip() in cupons_validos

# 1. RED: Escrevemos este teste primeiro!
def test_deve_aceitar_cupom_valido_com_espacos_e_minusculo():
    # Passamos um dado bagunçado para forçar a regra de negócio
    resultado = validar_cupom(" senai20 ")
    assert resultado is True
```

D. Responda:
1. Qual é o perigo de pular o passo "Red" e ir direto para a codificação da funcionalidade?
2. Por que o passo "Refactor" é considerado o momento de cuidar da qualidade do código?
3. O que significa quando um teste TDD falha antes de você mexer no código de produção?
4. Pense e responda, qual seria a assinatura do teste para validar um *cupom vencido*?
5. Como o TDD ajuda os membros do time a entenderem como usar a função que você criou?

Isolamento e Anatomia do Teste (Padrão AAA)

A. Conceito: Para que um teste seja confiável, ele precisa ser como um experimento de laboratório: **isolado** de influências externas e **organizado** para que qualquer pessoa entenda o que está sendo testado.

Vamos entender por que o isolamento é vital e como o padrão **AAA** coloca ordem na casa.

O Problema do "Vizinho Barulhento" (Isolamento)

Imagine que você tem dois testes: o Teste A cria um usuário e o Teste B deleta um usuário. Se o Teste B rodar antes do Teste A, ele pode tentar deletar algo que ainda não existe e falhar. Isso é um **teste instável (flaky test)**.

- **Independência:** Um teste nunca deve precisar que outro teste passe primeiro.
- **Limpeza:** Se o teste criou um dado no banco, ele (ou o sistema) deve apagar esse dado ao terminar. O próximo teste deve encontrar um "campo limpo".
- **Determinismo:** Se você rodar o teste 1.000 vezes, o resultado deve ser o mesmo em todas elas.

A Anatomia do Teste: O Padrão AAA

Para evitar que o código do teste vire uma bagunça ilegível, usamos o padrão **AAA**, que divide o teste em três blocos claros:

- **Arrange (Preparar)**

É aqui que você "monta o palco". Você prepara tudo o que o teste precisa para rodar.

  - **O que acontece:** Define variáveis, cria objetos, prepara o banco de dados e define os dados de entrada.
  - **Exemplo:** "Vou testar uma calculadora. Primeiro, eu instancio a calculadora e defino os números 5 e 10."

- **Act (Agir)**

É o momento da ação principal. Você executa a funcionalidade que quer testar.

  - **O que acontece:** Geralmente é **uma única linha** de código. É o gatilho.
  - **Exemplo:** "Eu mando a calculadora somar os dois números."

- **Assert (Verificar)**

É a hora da verdade. Você compara o resultado obtido com o resultado esperado.

  - **O que acontece:** O teste faz uma pergunta: "O resultado foi 15?". Se sim, o teste passa. Se não, ele falha.

- **Exemplo:** "Verifico se o retorno da soma é igual a 15."

Por que isso é importante?
Seguir essa anatomia transforma o código do teste em uma **documentação técnica**.

- **Manutenção fácil:** Se um teste falhar, você sabe exatamente se o problema foi na preparação dos dados (**Arrange**), na execução da lógica (**Act**) ou se o resultado apenas veio diferente do esperado (**Assert**).
- **Leitura rápida:** Qualquer desenvolvedor novo na equipe consegue ler o teste e entender o cenário em segundos, sem precisar decifrar uma "sopa de letrinhas".

Dica de Ouro: Se o seu bloco **Act** estiver ficando muito grande ou com muitas ações, talvez você esteja tentando testar coisas demais em um único teste. O ideal é: **Um objetivo por teste.**

B. Visualizando:

C. Exemplo de Código:

Python
```
def aplicar_desconto(valor_carrinho, valor_desconto):
    return max(0, valor_carrinho - valor_desconto)
#Uma função que retira um valor do total, limitando a zero para não dar saldo negativo.

def test_aplicar_desconto_nao_deve_resultar_em_valor_negativo():
    # ARRANGE (Delimitador visual organizando o pensamento.)
    carrinho_atual = 50.0
    cupom_gigante = 200.0
    valor_esperado = 0

    # ACT (Momento da ação única e objetiva.)
    resultado = aplicar_desconto(carrinho_atual, cupom_gigante)

    # ASSERT (A checagem da verdade.)
```

```
assert resultado == valor_esperado
```

D. Responda:
1. Por que é uma má prática ter mais de um "Act" (chamadas a diferentes métodos principais) no mesmo teste?
2. Como o padrão AAA facilita encontrar o defeito quando um colega lê seu código meses depois?
3. Oque ocorre se modificar o código do `Arrange` para testar um carrinho de `100.0` com desconto de `20.0`.?
4. O que é um teste *Flaky* e como o isolamento de dados do "Arrange" resolve isso?
5. Explique a relação do TDD com o padrão AAA. Eles podem (ou devem) ser usados juntos?

FAQ - Dúvidas Reais

Pergunta: No dia a dia da empresa, não tem tempo para TDD. O que eu faço?

> **Resposta:** A pressa é inimiga do código sustentável. Você pode negociar prazos mostrando que testes não são "trabalho extra", mas sim prevenção de *bugs* que parariam o servidor de produção à noite. Use as heurísticas para priorizar e automatizar primeiramente o "núcleo de negócio" vital.

Pergunta: Teste exploratório exige documentação pesada?

> **Resposta:** Não! Em vez de escrever centenas de casos de uso (planilhas fixas), usamos sessões focadas e documentamos apenas os "Mapas Mentais" e os bugs críticos encontrados (por meio de ferramentas de registro de defeito).

Pergunta: O Pytest é melhor que a biblioteca *unittest* nativa do Python?

> **Resposta:** Ambos resolvem o problema, mas a adoção do `Pytest` pela indústria se deve à sua simplicidade (uso direto do `assert`), falta de necessidade de criar classes complexas para testes simples, e o poder das *fixtures* no isolamento do código.

Pergunta: Como a Inteligência Artificial pode me ajudar no TDD?

> **Resposta:** A IA é uma aliada formidável na criação da *massa de dados* (o Arrange). Você pode pedir para IA: *"Gere 10 exemplos de JSON de entrada mal formatada para minha API"*. A ressalva ética: nunca envie credenciais reais de banco ou chaves de segurança da sua empresa no prompt.

Pergunta: Meus testes estão instáveis (Passam na minha máquina, mas falham no servidor). Onde olho?

> **Resposta:** Isso é um alerta vermelho de vazamento de estado (*flaky test*). Revise o isolamento (o padrão AAA): certifique-se de que nenhum teste depende do banco de dados deixado sujo pelo teste anterior e se certifique de que não está dependendo da data e hora real do sistema operacional.

Exercícios de Fixação

1. Identificação do Red-Green-Refactor

Descrição: Observe seu próprio processo de codificação de um script que você fez recentemente. Escreva como seria o fluxo TDD se você o tivesse utilizado (Qual seria a falha vermelha inicial? Qual seria a solução verde? Onde haveria refatoração?).
Entrega: Um parágrafo relatando o cenário.

2. Anatomia AAA
Descrição: Crie um código matemático simples de raiz quadrada, divida o caso de teste em blocos comentados de *Arrange*, *Act* e *Assert*.
Entrega: Trecho de código comentado (3 a 5 linhas).

3. Sessão Exploratória
Descrição: Acesse o Portal do Aluno do SENAI. Invista 10 minutos testando apenas com base em "intuição" e heurísticas extremas.
Entrega: Relatório bullet-point anotando três cenários de borda que você tentou forçar (ex: pesquisar por um caractere asiático no campo de CEP).

4. TDD Reverso
Descrição: Escreva o teste (com `assert`) validando uma função `eh_maior_de_idade(idade)`. Programe o teste para garantir que a idade 18 retorne `True` e 17 retorne `False`.
Entrega: Código do teste completo. Não precisa fazer a função em si ainda!

5. Caçando Flaky Tests
Descrição: Imagine que um teste de integração de banco falha de forma randômica aos finais de semana. Qual regra do Isolamento está sendo ferida?
Entrega: Breve texto (2 linhas) justificando e indicando como usar o *Arrange* para limpar o estado antes de testar.

6. Pytest na Prática - Parte 1
Descrição: Programe e crie um arquivo `test_calculadora.py`. Implemente, usando TDD (falhe primeiro), uma função que receba uma lista de preços e devolva o total somado.
Entrega: Script python finalizado e comprovação do terminal (`pytest -v`).

7. Pytest na Prática - Casos Extremos
Descrição: Utilizando a mesma função de soma (Exercício 6), crie novos testes que apliquem os princípios do teste exploratório (Error Guessing). Teste o envio de uma lista vazia e de uma lista com valores negativos.
Entrega: Os novos blocos de testes adicionados ao script.

8. Refatoração de Código Oculto
Descrição: A função de calcular total tem um `for` gigante. Refatore o código no seu passo "Blue/Refactor" usando o comando nativo `sum()` do Python. Garanta que a suíte de testes te dê o "sinal verde".
Entrega: Print (ou cópia) do terminal mostrando os testes executados com sucesso após a refatoração.

9. Refatorando um Teste para AAA
Descrição: Pegue um teste confuso da internet (ou gerado por IA intencionalmente sujo) que mistura chamadas de banco de dados com `asserts` no meio de laços de repetição. Reescreva-o usando fielmente o esqueleto `Arrange, Act, Assert`.
Entrega: Código legado (sujo) x Código refatorado (limpo).

10. Integração IA e Documentação
Descrição: Pegue a suíte construída nas atividades anteriores e solicite que uma Inteligência Artificial gere uma documentação padronizada sobre "O que estes testes cobrem e quais regras protegem". Analise criticamente se a IA acertou o contexto de negócio.
Entrega: A documentação gerada e um parágrafo crítico de "Revisão Humana" sobre as saídas.

Fechamento

Conexão com a SA: Todos os tópicos abordados hoje convergem diretamente para a nossa SA. Nas nossas documentações de planejamento e no momento de diagnosticar correções no software, vocês não dependerão de tentar "adivinhar na sorte" onde estão os defeitos. A documentação gerada pelos métodos exploratórios validará os requisitos de robustez do sistema que vocês desenvolverão, e os scripts construídos em TDD formarão as evidências para a entrega final de que o software tem um "padrão de desempenho e eficiência" em conformidade técnica.

Conexão com a Próxima Aula:

Se hoje focamos na metodologia e anatomia lógica (o cérebro do teste e como isolá-lo), na nossa próxima aula nós mergulharemos na avaliação das características não funcionais do sistema, verificando usabilidade, experiência do usuário e critérios de segurança e performance.

Vamos trabalhar:

- Testes Não Funcionais (Performance, Carga, Estresse),
- Testes de Usabilidade e Acessibilidade Digital (UX) e os
- Fundamentos de Segurança da Informação nos testes.